

1 Prioritäten

1. Zuverlässiges, robustes und reproduzierbares Vorgehen zur Extraktion und Migration der notwendigen Informationen.
2. Inkrementelles Erweitern des Outputs der Felder um sich vorläufigem Ziel-Schema anzunähern. Grundlage hierfür eine zukunftsfähige und wartbare Lösung für kommerziellen Gebrauch.
3. Erarbeiten besonderer Herausforderungen der Datenkonsolidierung (z.B. fehlende Informationen, Inkonsistenzen, Abweichungen zwischen Standorten).
4. Transparente Dokumentation der Entscheidungen und Vorgehensweise, um die Nachvollziehbarkeit zu gewährleisten. Auflistung weiterer Potenziale und aktueller Leistungen.

2 Vorgehen: Ziel I

2.1 Exploration

- (I) Prüfen der Intra-Standort Variabilität der Eingaben (i.e. wie hoch ist die Eingabekonsistenz innerhalb der Standorte)
- (I) Prüfen der Inter-Standort Variabilität der Eingaben (i.e. wie hoch ist die Eingabekonsistenz zwischen den Standorten)
- Ziel: Gibt es Gemeinsamkeiten zwischen den Standorten die die eine einheitliche Extraktion von Informationen zulassen (required)?
- Ziel: Welche Abweichungen erschweren die Extraktion von Informationen?
- Ergebnis: Folgende Tabelle zeigt welche Informationen einheitlich aus allen Dateien zuverlässig ausgelesen werden können:

.html Quelle	Info
Titel (<h1> Tag)	Zimmer, Vermarktungsart, Objekttyp
<div class=„expose“ data-objekt-nr=„xxxx“>	Objekt-ID
Objekt-Pfad	Quelle
<p class=„lage“>„xxxx“</p>	Adresse
<table class=„eckdaten“>	Wohnflaeche

2.2 Umsetzung der Erkenntnisse

- Regex und Wortsuche aus auf <h1> als Primärquelle mit Fallback auf Eckdaten Tabelle. Warnungen und Errors werden geloggt und gesammelt mit einer Fehlerbeschreibung.
- Aufgrund von nicht eindeutig lösbaren Inkonsistenzen erwarten die fehlerhaften Outputs einen HITL Ansatz.

3 Vorgehen: Ziel II

- (II) Erwägen welches Datenmodell für den Kunden zukünftig am besten geeignet ist.
- Ziel: Vorschlag einer SQL-Datenbank für kommerziellen Gebrauch.
- Ergebnis: Erweitern des aktuellen Schemas um Informationen die für KundInnen und Vertrieb in einer SQL-Datenbank relevant sind

4 Nutzung KI

- (1)Initiale Vorschläge der KI berücksichtigten nicht wie Fehler im Output behandelt wurden und entschied eigenständig.
- Änderung (1): Transparenz durch seperate Ablage und Flagging von fehlerhaften Extraktionsprozessen (Abbildung 2, Abbildung ??)
- (2) KI schlägt Validierung des Outputs durch „jsonschema“ vor.

- Änderung (2): Validierung des Outputs durch „jsonschema“ alleine könnte zu starr sein, da die genau Ursache von Abweichungen in Größe und Art unklar bleibt. Stattdessen wird der Output in eine separate Datei geschrieben und mit einem Flag versehen, um die fehlgeschlagene Validierung zu kennzeichnen (3).

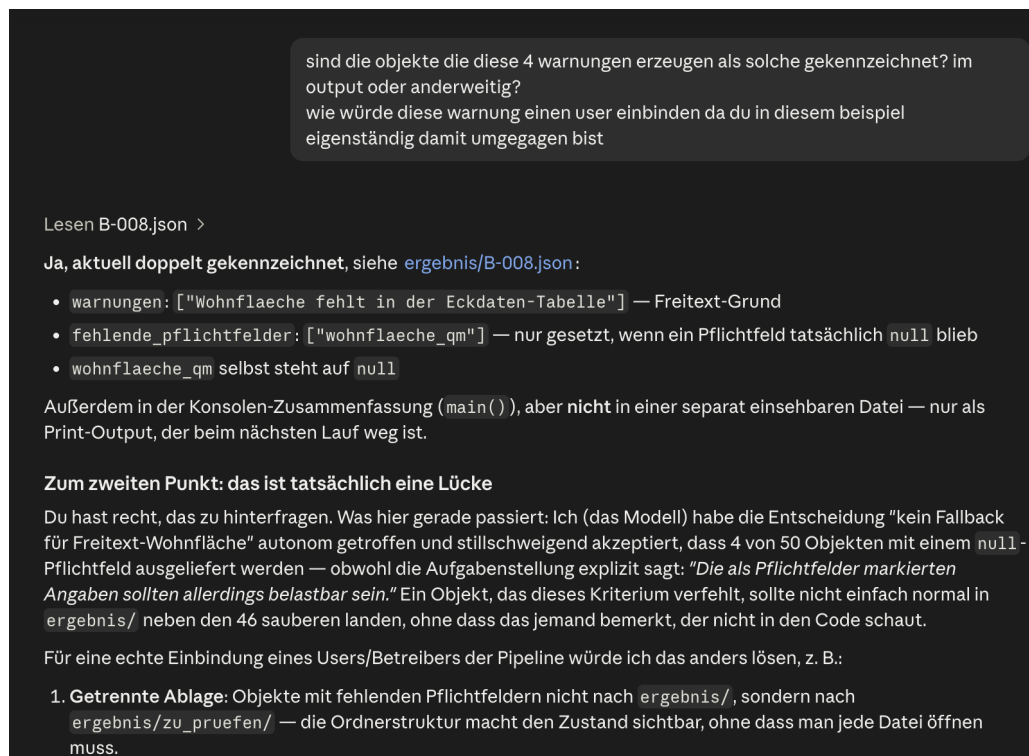


Abbildung 1: Error-Handling

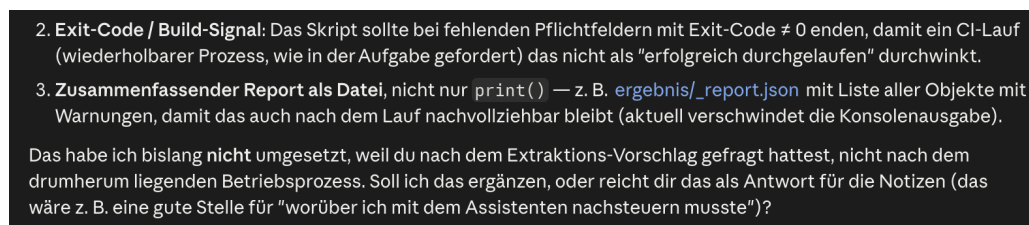


Abbildung 2: Error-Handling: Fortführung

Schwächen

- **Kein Freitext-Fallback für `wohnflaeche_qm`:** 4/50 Objekte landen aktuell mit `null` in einem Pflichtfeld in `zu_pruefen/` — bekannt, aber ungelöst.
- **Keine Prüfung auf doppelte `objekt_id`:** Da der Dateiname direkt aus `objekt_id` gebildet wird, würde ein zweites Objekt mit gleicher ID die erste Datei stillschweigend überschreiben — keine Warnung.
- **`_dezimal()` wird für Fläche und Zimmer gleichermaßen genutzt,** ohne Bereichsprüfung gegen die Schema-Grenzen (`maximum: 50` bei Zimmer, `maximum: 5000` bei Fläche) — eine falsch zugeordnete Zahl würde nicht auffallen.
- **Keine echte Validierung gegen `immobilien_schema.json`:** `fehlende_plichtfelder()` ist eine von Hand parallel gepflegte Kopie der Pflichtfeld-Logik. Ändert sich das Schema (z. B. neues Pflichtfeld), muss das im Code manuell nachgezogen werden — sonst diverging still.

Abbildung 3: Validierung: jsonschema